

Memoria Técnica - Learnify

Proyecto Integrador Profesional - Desarrollo de Aplicaciones Web

1. Introducción y Objetivos

Learnify es una plataforma educativa para aprender programación y desarrollo web. El objetivo principal es crear un sistema moderno que permita a los estudiantes explorar cursos, filtrar por categoría y dificultad, y acceder a contenido educativo de calidad.

El proyecto demuestra la aplicación de conceptos avanzados de JavaScript, arquitectura de software desacoplada, y buenas prácticas de desarrollo web profesional.

2. Planificación y Metodología

Arquitectura Elegida: Headless CMS

Se eligió una arquitectura desacoplada donde el backend (Strapi) gestiona los datos y el frontend (Astro) renderiza el contenido. Esta separación proporciona flexibilidad, escalabilidad y mejor rendimiento.

Ventajas de esta arquitectura:

El frontend se genera estáticamente en build time, mejorando significativamente el SEO y el rendimiento. El backend puede modificarse sin afectar el frontend. La base de datos está protegida detrás de una API REST. Los cambios en el contenido se reflejan automáticamente en el sitio.

Tecnologías Seleccionadas

Frontend: Astro (generación estática con JavaScript mínimo). Backend: Strapi (CMS Headless con API REST automática). Base de datos: SQLite en desarrollo, PostgreSQL en producción. Estilos: CSS con variables globales. Control de versiones: Git con GitHub.

Fases de Desarrollo

Fase 1: Diseño de la arquitectura y estructura de datos. Fase 2: Creación de colecciones en Strapi. Fase 3: Desarrollo del frontend con Astro. Fase 4: Implementación de funcionalidades (búsqueda, filtros, paginación). Fase 5: Optimización SEO y rendimiento. Fase 6: Documentación y despliegue.

3. Diseño y Tecnologías Empleadas

Estructura de Base de Datos

La base de datos relacional contiene seis colecciones principales:

Categories almacena las categorías de cursos (Desarrollo Web, Diseño, etc.). **Instructors** contiene los perfiles de profesores con información de contacto y redes sociales. **Courses** representa los cursos principales, cada uno con categoría, instructor y lecciones asociadas. **Lessons** son las unidades individuales dentro de cada curso, ordenadas secuencialmente. **BlogPosts** contiene artículos educativos con autor y categoría. **ContactSubmissions** almacena los mensajes del formulario de contacto.

Las relaciones entre tablas son principalmente 1:N (una categoría tiene muchos cursos, un instructor crea muchos cursos). Se implementó integridad referencial con ON DELETE CASCADE para mantener la consistencia de datos.

Conceptos JavaScript Aplicados

Desestructuración: Se utiliza para extraer propiedades de objetos de forma concisa, mejorando la legibilidad del código.

```
const { attributes } = course;
const { title, description } = attributes;
```

Array Methods: El proyecto implementa `map` para transformar datos, `filter` para búsquedas y filtros, `sort` para ordenamiento, y `slice` para paginación.

```
const beginnerCourses = courses.filter(c => c.attributes.difficulty ===
'beginner');
const sorted = courses.sort((a, b) =>
a.attributes.title.localeCompare(b.attributes.title));
const page = courses.slice(0, 6);
```

Async/Await: Todas las llamadas a la API de Strapi utilizan async/await para código asíncrono legible.

```
export async function fetchCourses() {
  const response = await fetch(`${STRAPI_URL}/api/courses`);
  const data = await response.json();
  return data.data;
}
```

Try/Catch: El manejo de errores se implementa en todas las operaciones asíncronas, previniendo que la aplicación se bloquee.

```
try {
  const response = await fetch(url);
  if (!response.ok) throw new Error(`Error: ${response.status}`);
  return await response.json();
} catch (error) {
  console.error('Error:', error);
  return [];
}
```

Expresiones Regulares: Se utilizan para búsqueda y validación. La búsqueda es case-insensitive y busca en múltiples campos.

```
function searchCourses(courses, query) {  
  const regex = new RegExp(query, 'i');  
  return courses.filter(course =>  
    regex.test(course.attributes.title) ||  
    regex.test(course.attributes.description)  
  );  
}
```

Normalización de Texto: Se implementa slugify para generar URLs amigables automáticamente desde títulos.

```
function slugify(text) {  
  return text  
    .toLowerCase()  
    .normalize('NFD')  
    .replace(/[\u0300-\u036f]/g, '')  
    .replace(/[\^\\w\\s-]/g, '')  
    .replace(/\\s+/g, '-')  
    .trim();  
}
```

Validación de Formularios: La validación utiliza regex para email y validaciones de longitud mínima para otros campos.

```
function validateEmail(email) {  
  const regex = /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/;  
  return regex.test(email);  
}
```

Cálculo de Tiempo de Lectura: Se estima automáticamente basado en el conteo de palabras.

```
function calculateReadingTime(content) {  
  const words = content.split(/\s+/).length;  
  const minutes = Math.ceil(words / 200);  
  return minutes;  
}
```

Paginación: Se implementa dividiendo resultados en páginas de 6 elementos con navegación intuitiva.

```
function paginateCourses(courses, page = 1, pageSize = 6) {  
  const start = (page - 1) * pageSize;  
  const end = start + pageSize;  
  return {  
    courses: courses.slice(start, end),  
    totalPages: Math.ceil(courses.length / pageSize)  
  };  
}
```

4. Sistema de Estilos

Arquitectura CSS

Se implementa un sistema de variables CSS en `:root` que define todos los valores reutilizables. Esto permite cambiar el tema completo modificando solo las variables, sin tocar los componentes individuales.

Variables Principales:

Colores: color-primary (#2563eb), color-secondary (#64748b), bg-primary, bg-secondary, text-primary, text-secondary, border.

Espaciado: spacing-xs (4px), spacing-sm (8px), spacing-md (16px), spacing-lg (24px), spacing-xl (32px).

Tipografía: font-sans (system fonts stack), tamaños de fuente para h1, h2, h3, p.

Dark Mode

El dark mode se implementa con una clase en el elemento html. Cuando el usuario activa dark mode, se aplica la clase “dark” que sobrescribe las variables CSS con valores oscuros.

```
html.dark {  
  --bg-primary: #0f172a;  
  --text-primary: #f1f5f9;  
}
```

Componentes Reutilizables

Se crearon componentes Astro reutilizables para Button y Card. Esto reduce duplicación de código y mantiene consistencia visual en toda la aplicación.

El componente Button soporta variantes (primary, secondary, outline), tamaños (sm, md, lg) y estados (hover, active, disabled).

El componente Card muestra contenido en formato de tarjeta con imagen, título, descripción y enlace.

Responsive Design

El diseño utiliza mobile-first con media queries en 768px (tablet) y 1024px (desktop). Todos los elementos se adaptan fluidamente a diferentes tamaños de pantalla.

5. Estrategia SEO Aplicada

SEO On-Page

Cada página tiene un H1 único y descriptivo. Los meta tags (title y description) se generan dinámicamente desde Strapi para cada curso y artículo.

Las URLs son amigables usando slugs generados automáticamente. Ejemplo: `/cursos/introduccion-a-javascript`.

Se incluye texto alternativo descriptivo en todas las imágenes. Los enlaces internos conectan coherentemente las páginas del sitio.

SEO Técnico

El archivo robots.txt se genera dinámicamente indicando a los buscadores qué pueden rastrear.

El sitemap.xml se genera automáticamente en build time con todas las páginas del sitio.

El HTML es semántico utilizando header, main, footer, article y otras etiquetas apropiadas.

Se incluyen Open Graph tags para mejorar la apariencia al compartir en redes sociales.

Contenido Relevante

El contenido está redactado de forma natural y humanizada, sin patrones de IA. Se evita el exceso de palabras clave forzadas. Las descripciones son claras y útiles para el usuario.

6. Funcionalidades Implementadas

Búsqueda Avanzada

La búsqueda utiliza expresiones regulares para buscar en múltiples campos (título, descripción). Es case-insensitive y se ejecuta en tiempo real sin recargar la página.

Filtros Dinámicos

Los usuarios pueden filtrar por dificultad (principiante, intermedio, avanzado) y por categoría. Los filtros se combinan lógicamente (AND).

Paginación

El listado de cursos está paginado con 6 elementos por página. La navegación es intuitiva con botones anterior/siguiente y números de página clickeables.

Validación de Formularios

El formulario de contacto valida todos los campos con feedback visual. Se valida email con regex, nombre mínimo 3 caracteres, asunto mínimo 5 caracteres, mensaje mínimo 10 caracteres.

Se requiere checkbox de consentimiento RGPD.

API Externa Integrada

La página de recursos consume la GitHub API en tiempo real para mostrar repositorios educativos. Busca repositorios etiquetados con "javascript-tutorial" ordenados por popularidad.

Muestra información como estrellas, forks e issues abiertos. Incluye manejo de errores con try/catch.

7. Seguridad y Cumplimiento

Validación de Entradas

Todas las entradas de usuario se validan en cliente con regex y en servidor con Strapi. Se previene inyección de código y XSS.

Protección de Datos

Strapi utiliza ORM que previene inyecciones SQL automáticamente. Las variables de entorno protegen secretos (JWT, API keys).

RGPD

El formulario de contacto recopila datos personales (nombre, email). Se requiere consentimiento informado explícito. Se documenta la política de privacidad. Se implementan medidas de seguridad básicas.

Autenticación

Strapi proporciona autenticación con JWT. El panel de administración requiere credenciales. Se implementan roles de usuario (Admin, Editor, Usuario, Invitado).

8. Conclusión

Learnify demuestra la aplicación profesional de conceptos avanzados de JavaScript, arquitectura moderna de software, y buenas prácticas de desarrollo web. El proyecto cumple todos los requisitos mínimos y proporciona una base sólida para futuras extensiones.

La arquitectura desacoplada permite escalabilidad, el código es reutilizable y mantenible, y la documentación es completa para facilitar el mantenimiento futuro.

9. Instrucciones de Instalación

Requisitos

Node.js 18+, pnpm, Git.

Instalación Local

Clonar el repositorio:

```
git clone https://github.com/Angelrp2/learnify.git
cd learnify
```

Instalar Strapi:

```
cd strapi
pnpm install
cp .env.example .env
pnpm run dev
```

En otra terminal, instalar Astro:

```
cd astro
pnpm install
cp .env.example .env
pnpm run dev
```

Acceder a <http://localhost:3000> para ver la plataforma.

Acceso a Strapi

Acceder a <http://localhost:1337/admin> para gestionar contenido.

En la primera ejecución, crear una cuenta de administrador.

Crear categorías, instructores y cursos de prueba.

10. Repositorio y Recursos

Repositorio GitHub: <https://github.com/Angelrp2/learnify>

Documentación adicional disponible en la carpeta docs/:

- ARQUITECTURA.md
- CONTENIDOS_TECNICOS.md
- ESTILOS.md
- SEO.md
- RGPD_SEGURIDAD.md
- DIAGRAMA_ER.md
- MANUAL_INSTALACION.md